

Access Specifiers in C# with Examples

In this article, I am going to discuss the **Access Specifiers in C#** with Examples. Please read our previous article before proceeding to this article where we discussed [Garbage Collection in .NET Framework](#) with Examples. As part of this article, we are going to discuss the following pointers which are related to the C# access specifiers.

1. [What are Access Specifiers in C#?](#)
2. [What are the Different Types of Access Specifiers Supported by C#.NET?](#)
3. [Understanding Type and Type Members in C#.](#)
4. [Understand Private, Public, Protected, Internal, Protected Internal, and Private Protected Access Specifiers in C# with Examples.](#)

What are Access Specifiers in C#?

Every keyword that we use such as private, public, protected, virtual, sealed, partial, abstract, static, base, etc. is called Modifiers. Access Specifiers are special kinds of modifiers using which we can define the scope of a type and its members.

So, in simple words, we can say that the Access Specifiers are used to define the scope of the type ([Class](#), [Interface](#), [Structs](#), [Delegate](#), [Enum](#), etc.) as well as the scope of their members ([Variables](#), [Properties](#), [Constructors](#), and [Methods](#)). Scope means accessibility or visibility that is who can access them and who cannot access them are defined by the Access Specifiers. See, I have a class with a set of members, who can consume these members, and who cannot consume these members are defined by the access specifiers.

Different Types of Access Specifiers in C#:

C# supports 6 types of access specifiers. They are as follows

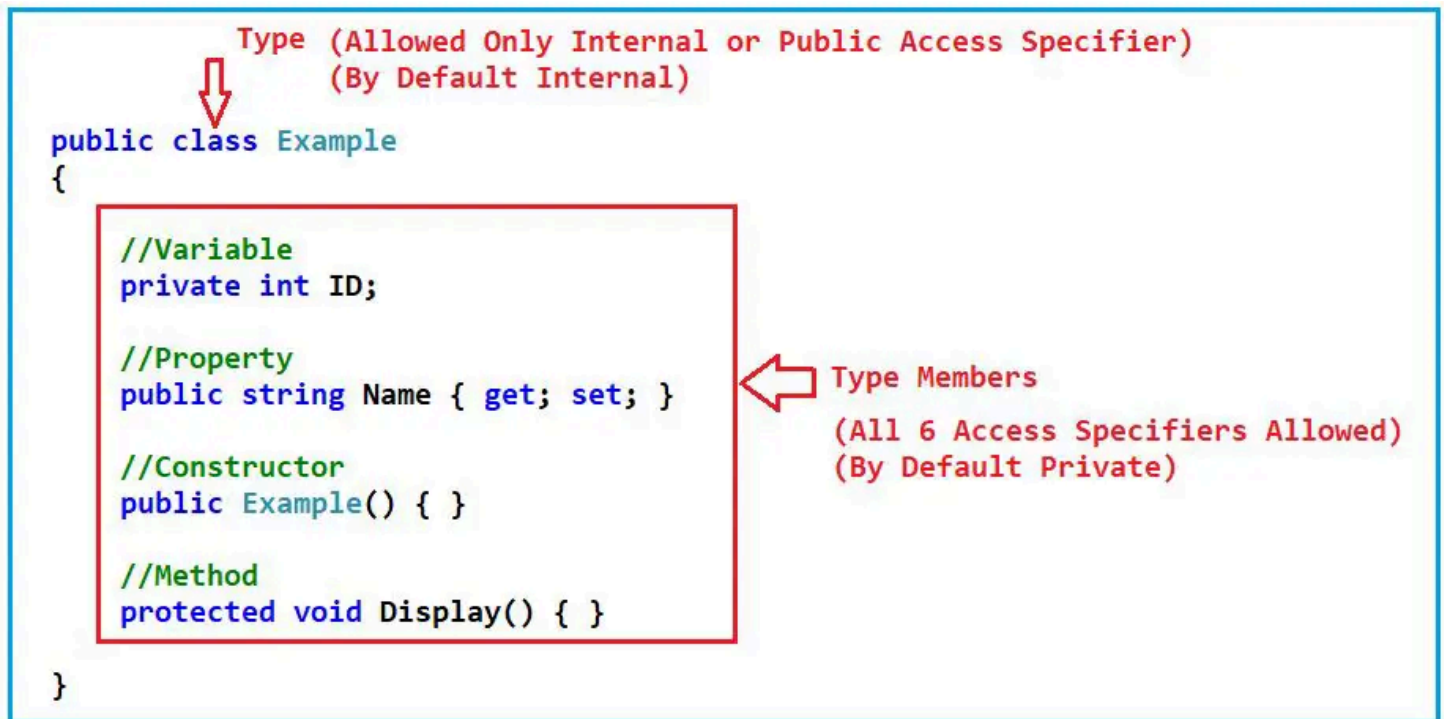
1. [Private](#)
2. [Public](#)
3. [Protected](#)
4. [Internal](#)
5. [Protected Internal](#)
6. [Private Protected](#) (C# Version 7.2 onwards)

Members that are defined in a type with any scope or specifiers are always accessible within that type; restriction comes into the picture only when we try to access them outside of the type. Each and every access specifier has a different scope and let us understand the scope of each access specifier with examples.

Note: I am going to explain the example using Visual Studio 2019. The previous version of Visual Studio might not be supporting the newly provided access specifier private protected.

Understand Type and Type Members in C#:

Before understanding Access Specifier, let us first understand what are Types and Type Members in C#. Please have a look at the below diagram. Here, Example (which is created by using the class keyword) is a Type, and Variable ID, Property Name, Constructor Example, and Method Display are type members.

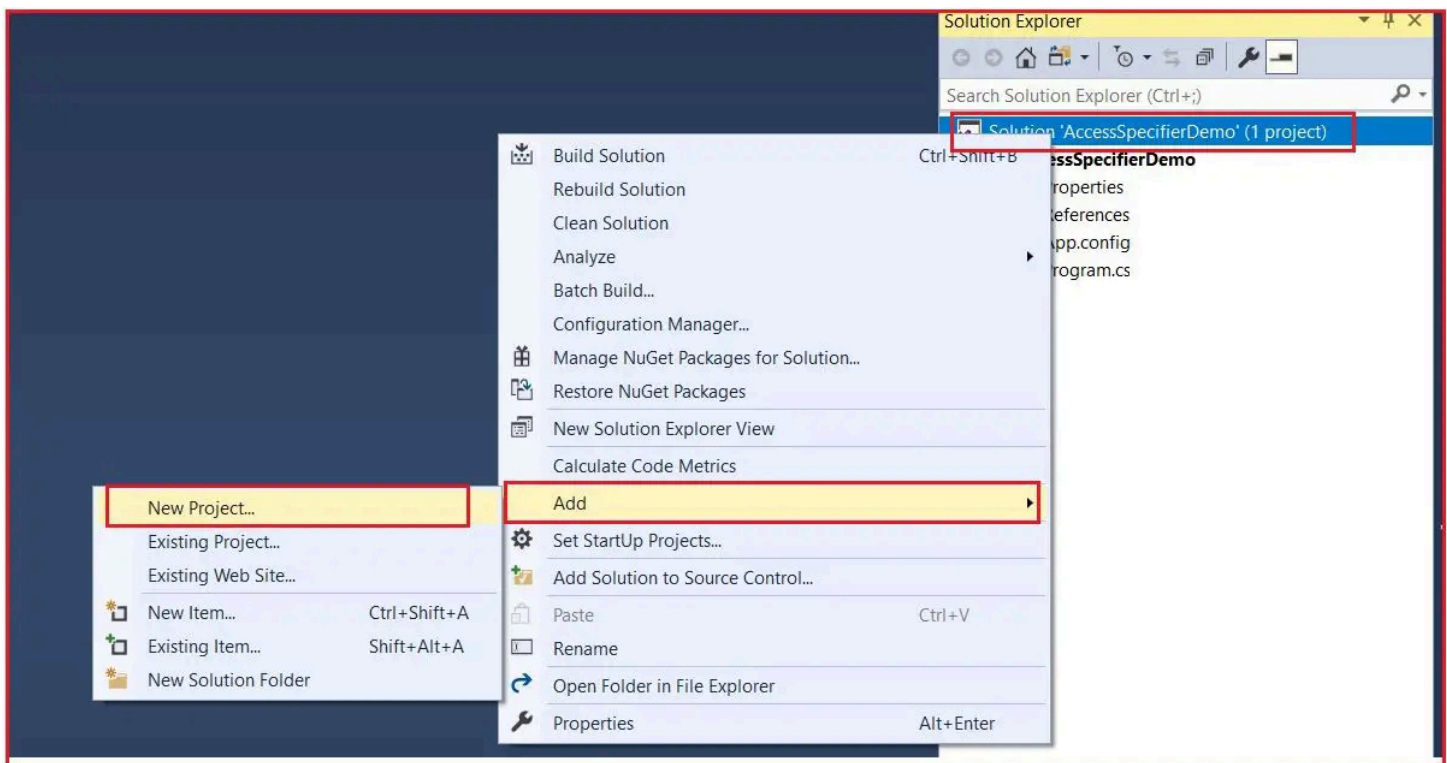


So, in general classes, structs, enums, interfaces, and delegates are called types, and variables, properties, constructors, methods, etc. that normally reside within a type are called type members. The type members can have all 6 access specifiers whereas types can have only 2 (internal, public) Access Modifiers. By default, if we have not specified any type, then for type is going to be internal access specifier and for type members, it is going to be a private Access Specifier. With this keep mind, let us proceed and understand all 6 Access Specifiers in C# with Examples.

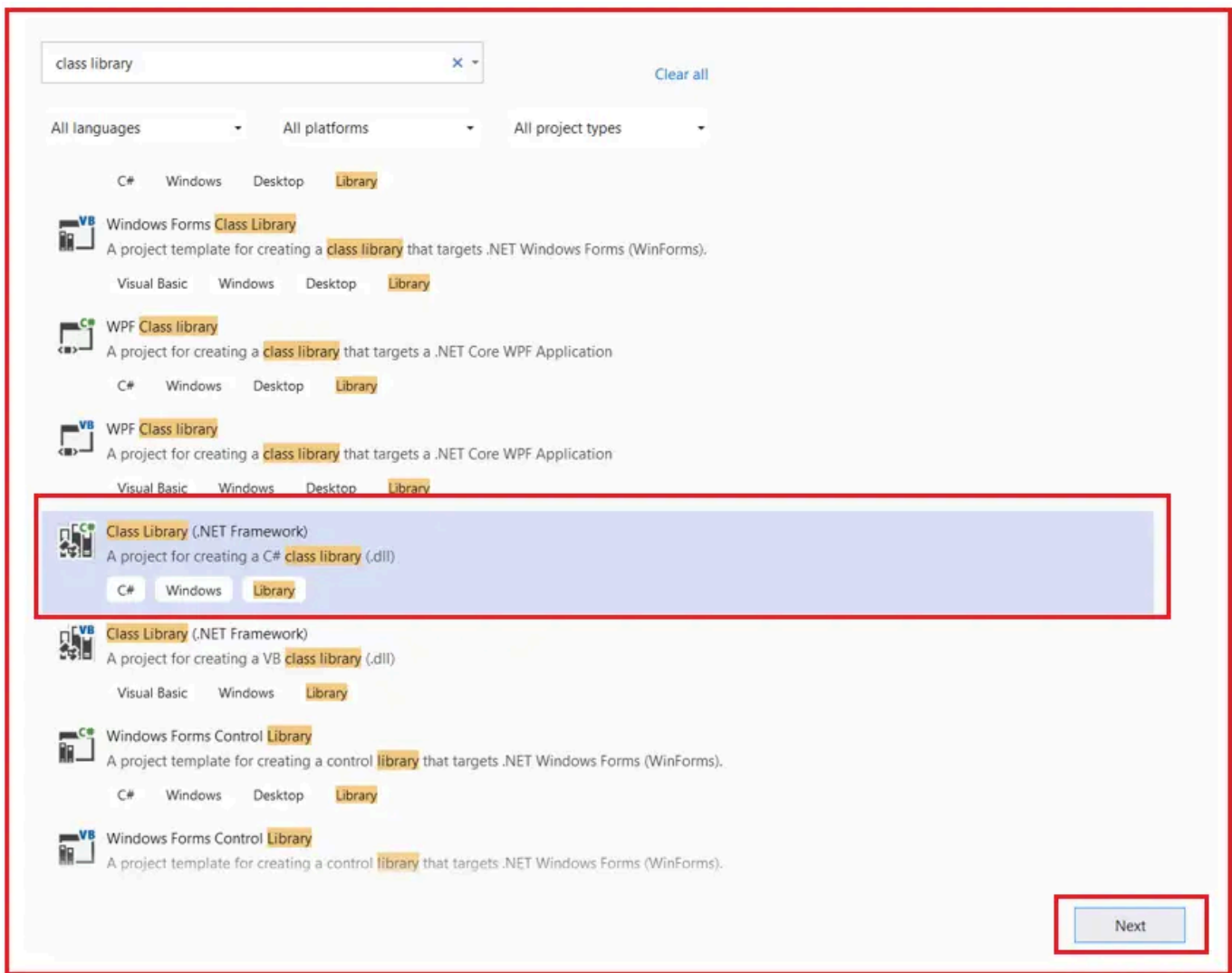
Example to Understand Access Specifiers in C# with Examples:

Let's discuss each Access Specifier in C# with Examples. For this create a new console application with the name **AccessSpecifierDemo**. Once you created this Console Application one assembly is created with the extension EXE. To understand Access Specifiers in C#, we at least need two assemblies. So, let us add a class library project to our solution which will create another assembly with a DLL extension. We need to follow the below steps to add the class library project.

Right-click on the Solution Explorer and then select **Add -> New Project** option from the context menu as shown in the below image.



Once you click on the New Project, it will open the following Add New Project Dialog Box. Here, first, search for the class library in the search window and then select **Class Library (.NET Framework)** using the C# language project template and then click on the Next button as shown in the below image.



Once you click on the OK button, it will open the Configure Your New Project window. Provide the Project name as AssemblyOne and select 4.8 as the Dot Net Framework and then click on the Create button as shown in the below image.

Configure your new project

Class Library (.NET Framework)

C#

Windows

Library

Project name

AssemblyOne

Location

D:\Projects\AccessSpecifierDemo

Framework

.NET Framework 4.8

Back

Create

Once you click on the Create button, it will add the Class Library Project with the name **AssemblyOne** to our solution. If you have followed the steps correctly, now you should have two projects in the solution explorer as shown in the below image.



Now build the solution, and you will see that 2 assemblies are generated. One DLL (for the Class Library Project with the name **AssemblyOne.DLL**) and one EXE (for the Console Application with the name **AccessSpecifierDemo.EXE**). To locate the assembly location i.e. where the assembly is generated, please follow the below steps.

1. Right-click on the AssemblyOne project (or your console application project), in solution explorer and select Open Folder in Windows Explorer.
2. Open bin folder
3. Then open the Debug folder

4. In the Debug folder, you should see **AssemblyOne.dll** or **AccessSpecifierDemo.exe**, which is the physical assembly.

What are Assemblies in .NET Framework?

According to MSDN, Assemblies are the building block of .NET Framework applications; they form the fundamental unit of deployment. In simple words, we can say that Assembly is nothing but a precompiled .NET Code that can be run by CLR (Common Language Runtime).

For a console application, the assembly is EXE and for the class library project, the assembly is DLL. We cannot run a DLL directly but we can run an EXE directly. For more detailed information about assemblies, please visit the following link where we explained Assemblies in Dot Net Framework.

Assemblies in Dot Net Framework

First, we will discuss Access Specifiers or Access Modifiers with the Type Members and then we will discuss Access Specifiers with the Type.

Access Specifiers or Access Modifiers with the Type Members:

Access Specifiers or Access Modifiers defined the scope of the type members. That is from where we can access them and from where we cannot access them. So, first of all, we need to understand what are the different scopes for type members. The different scope for the type members is as follows:

1. With the Class
2. Derived Class in Same Assembly
3. Non-Derived Class in Same Assembly
4. Derived Class in Other Assemblies
5. Non-Derived Class in Other Assemblies

Now, based on the Access Specifier, the restriction is applied to the Type Members. Now, let us proceed and understand the different access specifier and let us also understand from which scope we can access them.

Private Access Specifier or Modifier in C# with Examples:

When we declare a type member (variable, property, method, constructor, etc) as private, then we can access that member with the class only. From outside the class, we cannot access them.

Let us understand Private Members with an example. Now, go to the class library project and modify class1.cs class file as follows. As you can see, here we have created three classes and in the AssemblyOneClass1 we have created one private variable and then tried to access the private variable within the same class (AssemblyOneClass1), from the derived class (AssemblyOneClass2), and from the non-derived class (AssemblyOneClass3). And all these classes are within the same assembly only.

```
using System;
namespace AssemblyOne
{
    public class AssemblyOneClass1
    {
```

```

    private int Id;
    public void Display1()
    {
        //Private Member Accessible with the Containing Type only
        //Where they are created, they are available only within that type
        Console.WriteLine(Id);
    }
}
public class AssemblyOneClass2 : AssemblyOneClass1
{
    public void Display2()
    {
        //You cannot access the Private Member from the Derived Class
        //Within the Same Assembly
        Console.WriteLine(Id); //Compile Time Error
    }
}
public class AssemblyOneClass3
{
    public void Display3()
    {
        //You cannot access the Private Member from the Non-Derived Class
        //Within the Same Assembly
        AssemblyOneClass1 obj = new AssemblyOneClass1();
        Console.WriteLine(obj.Id); //Compile Time Error
    }
}
}

```

When you try to build the above code, you will get some compilation errors as shown in the below image. Here, you can see, it is clearly saying that you cannot access 'AssemblyOneClass1.Id' due to its protection level

	Code	Description
✖	CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level
✖	CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level

Here, in the above example, we are trying to access the private member from the same assembly i.e. within the class library project. Now, comment on those two statements which cause the compilation error. Once you comment on those statements which cause the compilation error in the class library project, then let us see what happens when we try to access the same private member from a different assembly. In our example, it is going to be our console application. For a better understanding, please modify the Program.cs class file as follows:

```

using System;
namespace AccessSpecifierDemo
{
    public class Program
    {

```

```

        static void Main(string[] args)
        {
        }
    }
    public class AnotherAssemblyClass1 : AssemblyOneClass1
    {
        public void Display4()
        {
            //You cannot access the Private Member from the Derived Class
            //from Other Assemblies
            Console.WriteLine(Id); //Compile Time Error
        }
    }
    public class AnotherAssemblyClass2
    {
        public void Display3()
        {
            //You cannot access the Private Member from the Non-Derived Class
            //from Other Assemblies
            AssemblyOneClass1 obj = new AssemblyOneClass1();
            Console.WriteLine(obj.Id); //Compile Time Error
        }
    }
}

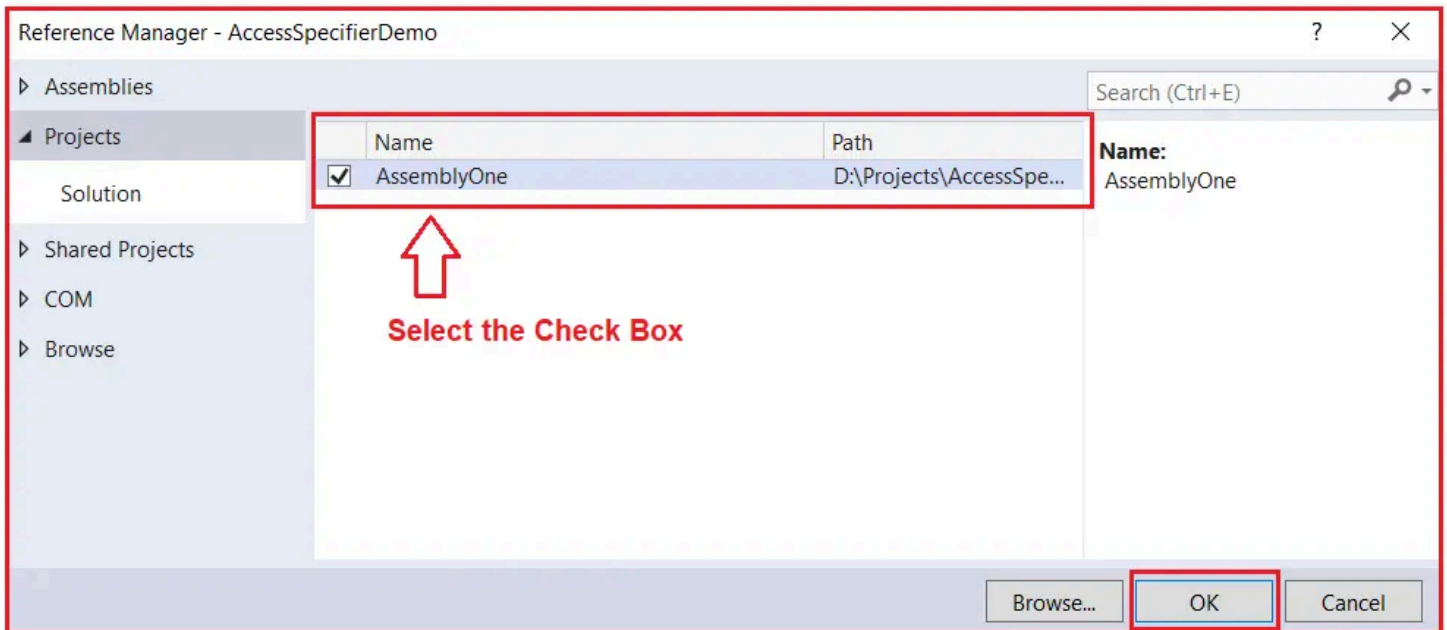
```

Now, you will get the following compilation error.

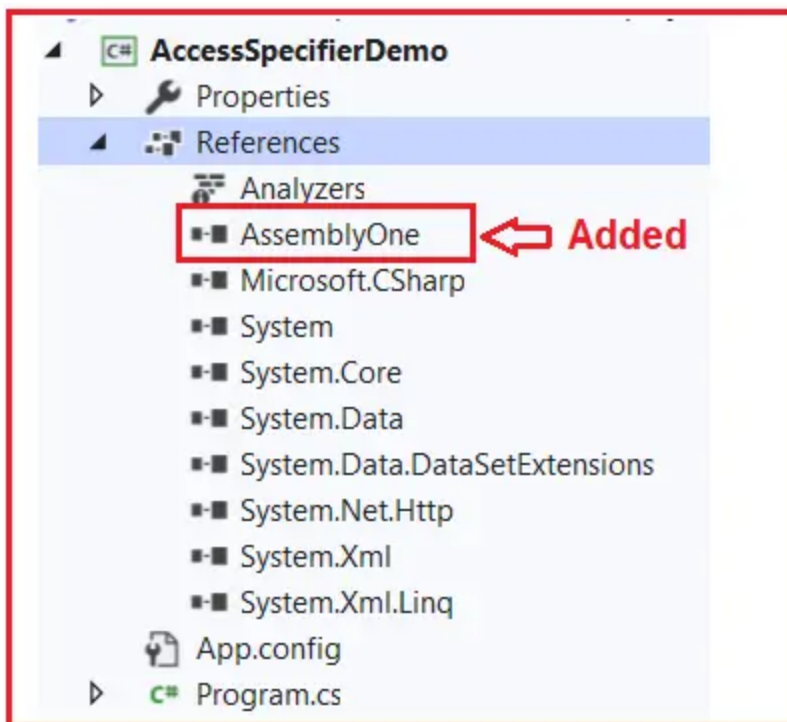
	Code	Description
✖	CS0246	The type or namespace name 'AssemblyOneClass1' could not be found (are you missing a using directive or an assembly reference?)
✖	CS0103	The name 'Id' does not exist in the current context
✖	CS0246	The type or namespace name 'AssemblyOneClass1' could not be found (are you missing a using directive or an assembly reference?)
✖	CS0246	The type or namespace name 'AssemblyOneClass1' could not be found (are you missing a using directive or an assembly reference?)

The above error is because of the AssemblyOneClass1 class file. We cannot access this class file directly from a different assembly. If you want to consume the members of this assembly, then first, you need to include or you need to add a reference to that assembly from the project where you want to access the members of this assembly. We want to consume our class library assembly in our console application, so, we need to add a reference to the class library project from our console application. To add assembly reference, please follow the below steps.

1. Expand the References folder under the AccessSpecifierDemo project, from Solution Explorer.
2. Right-click on the References folder and select Add Reference
3. From the Add Reference dialog box, select the Projects tab
4. From the list, select the AssemblyOne project and click on the OK button as shown in the below image.



Once you click on the OK button, you will see that AssrmblyOne dll should be added to the references folder as shown in the below image.



With the above changes in place, now include the namespace where the AssemblyOneClass1 is present. So, please modify the Program.cs class file as shown below to include the AssemblyOne namespace.

```
using AssemblyOne;
using System;
namespace AccessSpecifierDemo
{
    public class Program
    {
```

```

        static void Main(string[] args)
        {
        }
    }
    public class AnotherAssemblyClass1 : AssemblyOneClass1
    {
        public void Display4()
        {
            //You cannot access the Private Member from the Derived Class
            //from Other Assemblies
            Console.WriteLine(Id); //Compile Time Error
        }
    }
    public class AnotherAssemblyClass2
    {
        public void Display3()
        {
            //You cannot access the Private Member from the Non-Derived Class
            //from Other Assemblies
            AssemblyOneClass1 obj = new AssemblyOneClass1();
            Console.WriteLine(obj.Id); //Compile Time Error
        }
    }
}

```

With the above changes in place, now again build the project and this time you will get the following errors.

	Code	Description	Project ▲
✖	CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level	AccessSpecifierDemo
✖	CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level	AccessSpecifierDemo

These errors make sense that you cannot access the private members from derived and non-derived classes from different assemblies also. So, the scope of the private member in C#.NET is as follows:

1. **With the Class:** **YES**
2. **Derived Class in Same Assembly:** **NO**
3. **Non-Derived Class in Same Assembly:** **NO**
4. **Derived Class in Other Assemblies:** **NO**
5. **Non-Derived Class in Other Assemblies:** **NO**

Public Access Specifiers or Access Modifiers in C# with Examples:

When we declare a type member (variable, property, method, constructor, etc) as public, then we can access that member from anywhere. That means there is no restriction for public members.

Let's understand Public Members with an example. Please modify class1.cs file of the Class Library Project as follows. As you can see, we have modified the variable to public and then tried to access the public variable within the same class (AssemblyOneClass1), from the derived class (AssemblyOneClass2), and

from the non-derived class (AssemblyOneClass3). And all these classes are within the same assembly only and here we are not getting any compiler errors.

```
using System;
namespace AssemblyOne
{
    public class AssemblyOneClass1
    {
        public int Id;
        public void Display1()
        {
            //Public Members Accessible with the Containing Type
            //Where they are created
            Console.WriteLine(Id);
        }
    }
    public class AssemblyOneClass2 : AssemblyOneClass1
    {
        public void Display2()
        {
            //We Can access public Members from Derived Class
            //Within the Same Assembly
            Console.WriteLine(Id); //No-Compile Time Error
        }
    }
    public class AssemblyOneClass3
    {
        public void Display3()
        {
            //We Can access public Members from Non-Derived Classes
            //Within the Same Assembly
            AssemblyOneClass1 obj = new AssemblyOneClass1();
            Console.WriteLine(obj.Id); //No-Compile Time Error
        }
    }
}
```

Now, if you check the Program.cs class file of our console application, then you will see that we are not getting any errors as follows.

```
using AssemblyOne;
using System;
namespace AccessSpecifierDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
        }
    }
    public class AnotherAssemblyClass1 : AssemblyOneClass1
    {
    }
}
```

```

{
    public void Display4()
    {
        //We Can access the public Member from Derived Classes
        //from Other Assemblies
        Console.WriteLine(Id); //No-Compile Time Error
    }
}
public class AnotherAssemblyClass2
{
    public void Display3()
    {
        //We Can access the public Member from Non-Derived Classes
        //from Other Assemblies
        AssemblyOneClass1 obj = new AssemblyOneClass1();
        Console.WriteLine(obj.Id); //No-Compile Time Error
    }
}
}

```

So, the scope of the public member in C#.NET is as follows:

1. **With the Class: YES**
2. **Derived Class in Same Assembly: YES**
3. **Non-Derived Class in Same Assembly: YES**
4. **Derived Class in Other Assemblies: YES**
5. **Non-Derived Class in Other Assemblies: YES**

Protected Access Specifier or Access Modifier in C# with Examples:

Protected Members in C# are available within the containing type as well as to the types that are derived from the containing type. That means protected members are available within the parent class (i.e. the containing type) as well as to the child/derived classes (classes derived from the containing type).

Let us understand this Protected Access Specifier in C# with an example. Now, modify class1.cs class file as follows: Here, we are modifying the variable from public to protected. Here, you can observe while accessing the protected member from the containing type and derived classes, we are not getting any errors. But we are getting compilation errors when we are trying to access the protected member from the non-derived class within the same assembly.

```

using System;
namespace AssemblyOne
{
    public class AssemblyOneClass1
    {
        protected int Id;
        public void Display1()
        {
            //protected Members Accessible with the Containing Type
            //Where they are created

```

```

        Console.WriteLine(Id);
    }
}
public class AssemblyOneClass2 : AssemblyOneClass1
{
    public void Display2()
    {
        //We Can access protected Member from Derived Classes
        //Within the Same Assembly
        Console.WriteLine(Id); //No-Compile Time Error
    }
}
public class AssemblyOneClass3
{
    public void Display3()
    {
        //We Cannot access protected Member from Non-Derived Classes
        //Within the Same Assembly
        AssemblyOneClass1 obj = new AssemblyOneClass1();
        Console.WriteLine(obj.Id); //Compile Time Error
    }
}
}

```

Output:

	Code	Description	Project
	CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level	AssemblyOne

Now, let us try to access the protected members from different assemblies. Modify the Program.cs class file as follows. From the other assembly, you can access the protected member from the derived class, but you cannot access it from the non-derived classes.

```

using AssemblyOne;
using System;
namespace AccessSpecifierDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
        }
    }
    public class AnotherAssemblyClass1 : AssemblyOneClass1
    {
        public void Display4()
        {
            //We Can access the Protected Member from Derived Classes

```

```

        //from Other Assemblies
        Console.WriteLine(Id); //No-Compile Time Error
    }
}
public class AnotherAssemblyClass2
{
    public void Display3()
    {
        //We Cannot access the Protected Member from Non-Derived Classes
        //from Other Assemblies
        AssemblyOneClass1 obj = new AssemblyOneClass1();
        Console.WriteLine(obj.Id); // Compile Time Error
    }
}
}

```

Output:

Code	Description	Project
CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level	AccessSpecifierDemo

So, the scope of the protected members in C#.NET is as follows:

1. **With the Class:** YES
2. **Derived Class in Same Assembly:** YES
3. **Non-Derived Class in Same Assembly:** NO
4. **Derived Class in Other Assemblies:** YES
5. **Non-Derived Class in Other Assemblies:** NO

Internal Access Specifier or Access Modifier in C# with Examples:

Whenever a member is declared with Internal Access Specifier in C#, then it is available anywhere within the containing assembly. It's a compile-time error to access an internal member from outside the containing assembly.

Let us understand this Internal Access Specifier in C# with an example. Now, modify class1.cs class file as follows: Here, we are modifying the variable from protected to internal. Here, you can observe while accessing the protected member from the containing type, derived classes, and non-derived classes within the same assembly, we are not getting any errors.

```

using System;
namespace AssemblyOne
{
    public class AssemblyOneClass1
    {
        internal int Id;
        public void Display1()
        {

```

```

        //internal Members Accessible with the Containing Type
        //Where they are created
        Console.WriteLine(Id);
    }
}
public class AssemblyOneClass2 : AssemblyOneClass1
{
    public void Display2()
    {
        //We can access internal Members from Derived Classes
        //Within the Same Assembly
        Console.WriteLine(Id); //No-Compile Time Error
    }
}
public class AssemblyOneClass3
{
    public void Display3()
    {
        //We can access internal Members from Non-Derived Classes
        //Within the Same Assembly
        AssemblyOneClass1 obj = new AssemblyOneClass1();
        Console.WriteLine(obj.Id); //No-Compile Time Error
    }
}
}

```

Now, let us try to access the internal members from a different assembly. Modify the Program.cs class file as follows. From the other assembly, you cannot access the internal member either from the derived classes or from the non-derived classes.

```

using AssemblyOne;
using System;
namespace AccessSpecifierDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
        }
    }
    public class AnotherAssemblyClass1 : AssemblyOneClass1
    {
        public void Display4()
        {
            //We cannot access the Internal Member from Derived Classes
            //from Other Assemblies
            Console.WriteLine(Id); //Compile Time Error
        }
    }
    public class AnotherAssemblyClass2
    {

```

```

    public void Display3()
    {
        //We cannot access internal Member from Non-Derived Classes
        //from Other Assemblies
        AssemblyOneClass1 obj = new AssemblyOneClass1();
        Console.WriteLine(obj.Id); //Compile Time Error
    }
}

```

Output:

	Code	Description	Project ▲
✖	CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level	AccessSpecifierDemo
✖	CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level	AccessSpecifierDemo

So, the scope of the internal members in C#.NET is as follows:

1. **With the Class: YES**
2. **Derived Class in Same Assembly: YES**
3. **Non-Derived Class in Same Assembly: YES**
4. **Derived Class in Other Assemblies: NO**
5. **Non-Derived Class in Other Assemblies: NO**

Protected Internal Access Specifier or Access Modifier in C#:

Protected Internal Members in C# can be accessed anywhere within the same assembly i.e. in which it is declared or from within a derived class from another assembly. So, we can think, it is a combination of Protected and Internal access specifiers. If you understood the Protected and Internal access specifiers, then this should be very easy to follow. Protected means, members can be accessed within the derived classes, and Internal means within the same assembly.

Let us understand this Protected Internal Access Specifier in C# with an example. Now, modify class1.cs class file as follows: Here, we are modifying the variable from internal to protected internal. Here, you can observe while accessing the protected internal member from the containing type, from the derived classes, and from the non-derived class within the same assembly, we are not getting any compilation error.

```

using System;
namespace AssemblyOne
{
    public class AssemblyOneClass1
    {
        protected internal int Id;
        public void Display1()
        {
            //protected internal Members Accessible with the Containing Type
            //Where they are created
            Console.WriteLine(Id);
        }
    }
}

```

```

    }
}
public class AssemblyOneClass2 : AssemblyOneClass1
{
    public void Display2()
    {
        //We can access protected internal Member from Derived Classes
        //Within the Same Assembly
        Console.WriteLine(Id); //No-Compile Time Error
    }
}
public class AssemblyOneClass3
{
    public void Display3()
    {
        //We can access protected internal Member from Non-Derived Classes
        //Within the Same Assembly
        AssemblyOneClass1 obj = new AssemblyOneClass1();
        Console.WriteLine(obj.Id); //No-Compile Time Error
    }
}
}

```

Now, let us try to access the protected internal members from a different assembly. Modify the Program.cs class file as follows. From other assemblies, you can access the protected internal member from the derived classes, but you cannot access from the non-derived classes.

```

using AssemblyOne;
using System;
namespace AccessSpecifierDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
        }
    }
    public class AnotherAssemblyClass1 : AssemblyOneClass1
    {
        public void Display4()
        {
            //We can access the protected internal Members from Derived Classes
            //from Other Assemblies
            Console.WriteLine(Id); //No-Compile Time Error
        }
    }
    public class AnotherAssemblyClass2
    {
        public void Display3()
        {
            //We cannot access protected internal Members from Non-Derived Classes

```

```

        //from Other Assemblies
        AssemblyOneClass1 obj = new AssemblyOneClass1();
        Console.WriteLine(obj.Id); //Compile Time Error
    }
}
}

```

Output:

	Code	Description	Project ▲
✖	CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level	AccessSpecifierDemo

So, the scope of the protected internal members in C#.NET is as follows:

1. **With the Class: YES**
2. **Derived Class in Same Assembly: YES**
3. **Non-Derived Class in Same Assembly: YES**
4. **Derived Class in Other Assemblies: YES**
5. **Non-Derived Class in Other Assemblies: NO**

Private Protected Access Specifier or Access Modifier in C# with Examples:

The private protected members are accessible within the class and within the derived class of the same assembly but cannot be accessed from another assembly.

Let us understand this Private Protected Access Specifier in C# with an example. Now, modify class1.cs class file as follows: Here, we are modifying the variable from protected internal to private protected. Here, you can observe while accessing the protected internal member from the containing type, and from the derived classes within the same assembly, we are not getting any compilation error. But we are getting compilation errors while trying to access the private protected members from the non-derived classes of the same assembly.

```

using System;
namespace AssemblyOne
{
    public class AssemblyOneClass1
    {
        private protected int Id;
        public void Display1()
        {
            //Private Protected Members Accessible with the Containing Type
            //Where they are created
            Console.WriteLine(Id);
        }
    }
    public class AssemblyOneClass2 : AssemblyOneClass1
    {

```

```

    public void Display2()
    {
        //We can access Private Protected Member from Derived Classes
        //Within the Same Assembly
        Console.WriteLine(Id); //No-Compile Time Error
    }
}
public class AssemblyOneClass3
{
    public void Display3()
    {
        //We cannot access Private Protected Member from Non-Derived Class
        //Within the Same Assembly
        AssemblyOneClass1 obj = new AssemblyOneClass1();
        Console.WriteLine(obj.Id); //Compile Time Error
    }
}
}

```

Output:

	Code	Description	Project
	CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level	AssemblyOne

Now, let us try to access the private protected members from a different assembly. Modify the Program.cs class file as follows. From other assemblies, you cannot access the private protected members either from the derived classes or from the non-derived classes.

```

using AssemblyOne;
using System;
namespace AccessSpecifierDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
        }
    }
    public class AnotherAssemblyClass1 : AssemblyOneClass1
    {
        public void Display4()
        {
            //We cannot access Private Protected Member from Derived Classes
            //from Other Assemblies
            Console.WriteLine(Id); //Compile Time Error
        }
    }
    public class AnotherAssemblyClass2

```

```

{
    public void Display3()
    {
        //We cannot access Private Protected Member from Non-Derived Class
        //from Other Assemblies
        AssemblyOneClass1 obj = new AssemblyOneClass1();
        Console.WriteLine(obj.Id); //Compile Time Error
    }
}

```

Output:

	Code	Description	Project
✖	CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level	AccessSpecifierDemo
✖	CS0122	'AssemblyOneClass1.Id' is inaccessible due to its protection level	AccessSpecifierDemo

So, the scope of the private protected members in C#.NET is as follows:

1. **With the Class: YES**
2. **Derived Class in Same Assembly: YES**
3. **Non-Derived Class in Same Assembly: NO**
4. **Derived Class in Other Assemblies: NO**
5. **Non-Derived Class in Other Assemblies: NO**

Note: Here, I have shown the example by using a variable, but the same is applicable to other members of a class like properties, methods, and constructors. The following table shows the summary of all the access specifiers with the type members.

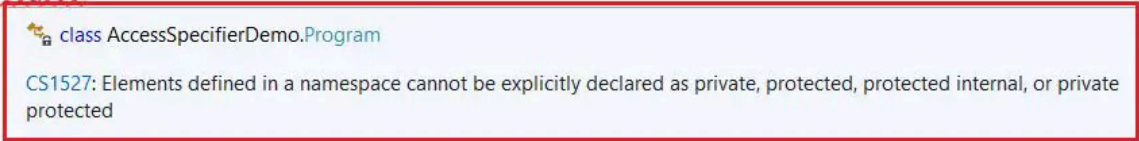
Caller's location	public	protected internal	protected	internal	private protected	private
Within the class	✓	✓	✓	✓	✓	✓
Derived class (same assembly)	✓	✓	✓	✓	✓	✗
Non-derived class (same assembly)	✓	✓	✗	✓	✗	✗
Derived class (different assembly)	✓	✓	✓	✗	✗	✗
Non-derived class (different assembly)	✓	✗	✗	✗	✗	✗

As of now, we have discussed how to use access specifiers with the type members. Now, let's see how to use the access specifiers in C# with the type.

Access Specifiers with Type in C#:

We can use all 6 access specifiers with type members in C# but type allows only two access specifiers i.e. Internal and Public. It is a compile-time error to use private, protected, protected internal, and private protected access specifiers with types. The following code will generate a compiler error (as we marked the class Program as private) stating **Elements defined in a namespace cannot be explicitly declared as private, protected, protected internal, or private protected**. So, the only allowed access specifiers for a type is internal and public and if we have not specified any access specifier, then by default it is going to be internal.

```
namespace AccessSpecifierDemo
{
    //Error: Elements defined in a namespace cannot be explicitly declared as private,
    //protected, protected internal, or private protected
    0 references
    private class Program
    {
        0 references
        public static void Main()
        {
            Console.WriteLine("This code will not compile");
        }
    }
}
```



Note: The point that you need to remember is if you want to access the class only within the same assembly, then you need to declare the class as internal and if you want to access the class from the same assembly as well as from other assemblies then you need to declare the class as public.

Example to Understand Internal and Public Access Specifier in C# with Type:

Please modify class1.cs class file of the class library project as follows: Here, we marked the class as internal and here you can see we are making inheritance relationships as well as, we are able to create an instance of the internal class within the same assembly

```
using System;
namespace AssemblyOne
{
    internal class AssemblyOneClass1
    {
        public int Id;
        public void Display1()
        {
            Console.WriteLine(Id);
        }
    }
}
```

```

internal class AssemblyOneClass2 : AssemblyOneClass1
{
    public void Display2()
    {
        Console.WriteLine(Id);
    }
}
internal class AssemblyOneClass3
{
    public void Display3()
    {
        AssemblyOneClass1 obj = new AssemblyOneClass1();
        Console.WriteLine(obj.Id);
    }
}
}

```

Now, let us try to use the internal class from a different assembly. Please modify the Program.cs class file as follows: Here, you can observe that we are unable to make the class inheritance as well as we are unable to create an instance of the internal class. Here, we are getting compilation errors.

```

using AssemblyOne;
using System;
namespace AccessSpecifierDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
        }
    }
    //You cannot make inheritance relationship because AssemblyOneClass1 is internal
    //Internal cannot be accessible to outside assembly
    public class AnotherAssemblyClass1 : AssemblyOneClass1
    {
        public void Display4()
        {
        }
    }
    public class AnotherAssemblyClass2
    {
        public void Display3()
        {
            //You cannot create an instance because AssemblyOneClass1 is not accessible
            //to outside assembly
            AssemblyOneClass1 obj = new AssemblyOneClass1();
        }
    }
}

```

Output:

	C..	Description	Project
✖	CS0122	'AssemblyOneClass1' is inaccessible due to its protection level	AccessSpecifierDemo
✖	CS0122	'AssemblyOneClass1' is inaccessible due to its protection level	AccessSpecifierDemo
✖	CS0122	'AssemblyOneClass1' is inaccessible due to its protection level	AccessSpecifierDemo

Now, modify the class from internal to public in class1.cs class file as shown in the below code. Once we make the class AssemblyOneClass1 public, all the compilation errors are gone.

```
using System;
namespace AssemblyOne
{
    public class AssemblyOneClass1
    {
        public int Id;
        public void Display1()
        {
            Console.WriteLine(Id);
        }
    }
    internal class AssemblyOneClass2 : AssemblyOneClass1
    {
        public void Display2()
        {
            Console.WriteLine(Id);
        }
    }
    internal class AssemblyOneClass3
    {
        public void Display3()
        {
            AssemblyOneClass1 obj = new AssemblyOneClass1();
            Console.WriteLine(obj.Id);
        }
    }
}
```

So, the point that you need to remember is, if you declare any type as internal then it is only accessible or available within the same assembly where it is created and if you created the type with public access specifier, then that type is available and accessible within the assembly where it is created as well as from other assemblies.

What are the default Access Specifiers in C# for a Class and Class Members?

If we don't specify an access specifier in C# then for Class, the default access specifier is internal and for class members it is private.

In the next article, I am going to discuss [Encapsulation in C#](#) with Examples. Here, in this article, I try to explain the Access Specifiers in C# using Type and Type Members with Examples. I hope you enjoy this article and understand the C# Access Specifiers. I would like to have your feedback. Please post your feedback, question, or comments about Access Specifiers in C# with Examples article.